

VRP 考核项目技术学习笔记

日期: 2026-07-25

本笔记用于理解 ZJU 的 ZJUI 学院 CVRP 考核项目。目标不是直接写最终报告，也不是直接实现 `solve.py`，而是把题目、数据、可行性规则、评价指标、baseline 和 AI-based 方法先讲清楚，使后续实现有稳定基础。

1. 用一句话理解这个项目

这个项目要求我们为校园快递配送任务设计一个 CVRP 求解器：车辆从同一个仓库点 depot 出发，服务所有客户点，每辆车不能超过容量限制，并且希望总行驶距离尽量短。

更具体地说，每个测试实例可以理解成“某一天校园内的包裹配送任务”：

- depot 是物流中心或出发点。
- customer nodes 是需要收包裹的地点。
- demand 是每个地点的包裹需求量。
- capacity 是单辆车最大可装载量。
- routes 是我们输出的多条车辆路径。
- cost 是所有路径从 depot 出发、访问客户、回到 depot 的总欧氏距离。

项目最终不是只看模型能不能输出东西，而是看输出的 routes 是否满足 CVRP 规则，以及在合法的前提下 cost、gap 和 inference time 表现如何。

2. CVRP 到底是什么

CVRP 是 Capacitated Vehicle Routing Problem，中文可以理解为“容量约束车辆路径问题”。

VRP 的核心问题是：有一批客户点需要服务，车辆从 depot 出发，应该怎么安排路径，使总路程尽量短。CVRP 在 VRP 上增加了容量约束：每辆车一次服务的客户 demand 总和不能超过 vehicle capacity。

本项目中的 CVRP 解由多条 route 组成。例如：

[[1, 7, 3], [2, 5, 4, 6]]

它表示：

depot -> 1 -> 7 -> 3 -> depot

depot -> 2 -> 5 -> 4 -> 6 -> depot

注意 depot 不写进 route 列表，因为它是隐含的。每条 route 都默认从 depot 出发，并最终回到 depot。

CVRP 的难点在于两个目标互相牵制：

- 如果每辆车只送很少客户，通常容易满足 capacity，但车辆路径数量多，总路程可能变长。
- 如果一辆车尽量多送客户，路径可能更短，但容易超过 capacity。
- 如果只贪心选择最近客户，局部看起来合理，但整体路线可能绕远。

因此 CVRP 是组合优化问题。客户数量从 50 增加到 100 后，可能路径组合数量会急剧增加，这也是公开测试集中加入 CVRP-100 的原因：它用来检查方法是否能泛化到更大的实例。

3. 本项目的数据应该怎么读

项目数据位于：

VRP_project/VRPData/

当前可见的三个数据文件是：

文件	内容	用途
train_data.pkl	7000 个 CVRP-50，带参考 routes 和 cost	训练、模仿学习、分析参考解
validation_data.pkl	1000 个 CVRP-50，带参考 routes 和 cost	调参、验证、比较 baseline
check_data_to_students.pkl	1000 个 CVRP-50 + 500 个 CVRP-100，无参考 routes 和 cost	公开测试、生成提交格式输出

官方隐藏测试集 check_data.pkl 不在当前文件列表中，但说明文档写明它的规模与公开测试集类似，也是 1000 个 CVRP-50 加 500 个 CVRP-100。最终 solve.py 要能处理隐藏测试集。

3.1 训练和验证数据的 tuple 结构

训练集和验证集中的每个实例是长度为 6 的 tuple:

(depot, loc, demand, capacity, routes, cost)

字段含义如下:

字段	含义	本项目中的形状或类型
depot	depot 坐标	[[x, y]], 形状是 (1, 2)
loc	客户点坐标	CVRP-50 中形状是 (50, 2)
demand	每个客户的需求量	CVRP-50 中长度是 50
capacity	单辆车容量	CVRP-50 中通常是 40.0
routes	OR-Tools 生成的参考路径	list[list[int]], 客户编号从 1 开始
cost	参考路径总 cost	float

本地检查得到:

- train_data.pkl: 7000 个实例, 全部是长度 6 tuple, 全部是 CVRP-50, capacity 为 40.0, demand 范围是 1 到 9。
- validation_data.pkl: 1000 个实例, 全部是长度 6 tuple, 全部是 CVRP-50, capacity 为 40.0, demand 范围是 1 到 9。
- 训练集参考 cost 平均约为 10.68, 验证集参考 cost 平均约为 10.60。
- 训练集参考解平均约有 6.80 条 route, 验证集参考解平均约有 6.80 条 route。

这些 routes 和 cost 是高质量参考标签, 不等于保证全局最优。说明文档写明这些标签由 OR-Tools 在每个实例 3000 ms 时间限制下生成。

3.2 公开测试数据的 tuple 结构

公开测试集中的每个实例是长度为 4 的 tuple:

(depot, loc, demand, capacity)

它没有 routes 和 cost。这是一个很重要的实现细节: 如果代码默认每个实例都是长度 6, 就会在测试集上直接报错。

本地检查得到：

- `check_data_to_students.pkl`：1500 个实例。
- 其中 1000 个是 CVRP-50, `capacity` 为 40.0。
- 其中 500 个是 CVRP-100, `capacity` 为 50.0。
- `demand` 范围是 1 到 9。

这说明后续方法不能只写死 50 个客户，也不能写死 `capacity` 为 40。应该根据每个实例的 `len(loc)` 和 `capacity` 动态处理。

3.3 客户编号和数组下标的区别

Python 列表下标从 0 开始，但项目输出中的客户编号从 1 开始。

例如：

- `loc[0]` 对应客户 1。
- `loc[7]` 对应客户 8。
- `route [8, 3, 10]` 表示访问 `loc[7]`、`loc[2]`、`loc[9]`。

这是最容易出错的地方之一。内部计算时可以用 0-based index，但输出 JSON 时必须转成 1-based customer id。

4. 什么样的 routes 才是合法解

一个 CVRP 解是否可行，比 `cost` 更基础。如果解不可行，即使 `cost` 很低，也不能算真正解决了问题。

本项目中的合法 routes 至少要满足下面这些条件。

4.1 每个客户必须恰好访问一次

对于 CVRP-50，合法解必须覆盖客户 1 到 50。对于 CVRP-100，合法解必须覆盖客户 1 到 100。

不能出现：

- 漏掉某个客户。
- 同一个客户出现在多条 route 中。
- 同一个客户在同一条 route 里重复出现。
- 出现超过客户数量范围的编号，例如 CVRP-50 中出现客户 51。

4.2 每条 route 不能超过 capacity

每条 route 的 demand 总和必须小于等于 capacity。

例如 capacity 是 40.0, 某条 route 是:

[1, 7, 3]

则需要计算:

$\text{demand}[0] + \text{demand}[6] + \text{demand}[2] \leq 40.0$

如果超过 40, 这条 route 就不可行。

4.3 depot 不写进 routes

depot 是隐含的, 不应该写成客户编号。合法 route 只包含客户 id。

正确:

[8, 3, 10]

含义:

depot -> 8 -> 3 -> 10 -> depot

错误思路:

[0, 8, 3, 10, 0]

因为输出格式中 depot 不应该出现在 route 列表里。

4.4 空 routes 只用于无法产生结果的情况

官方格式允许某个实例输出:

```
{"instance_id": 1, "routes": []}
```

但这表示没有有效结果。为了 feasibility rate, 正常情况下不应该主动输出空 routes。baseline 的第一目标应该是对每个实例都生成完整、合法的 routes。

5. 评价指标怎么理解

说明文档要求关注四类指标：

- Average Total Route Cost
- Feasibility Rate
- Average Gap to a Reference Baseline
- Average Inference Time

5.1 Average Total Route Cost

这是平均总路径代价，越小越好。

对单个实例来说，cost 是所有 route 的路径距离之和。每条 route 都从 depot 出发，访问 route 中的客户，再回到 depot。

如果 route 是：

[8, 3, 10]

则这条 route 的距离是：

$\text{dist}(\text{depot}, 8) + \text{dist}(8, 3) + \text{dist}(3, 10) + \text{dist}(10, \text{depot})$

这里的距离一般使用原始二维坐标上的欧氏距离。

Average Total Route Cost 就是对测试集中所有实例的 total cost 求平均。

5.2 Feasibility Rate

Feasibility Rate 是可行解比例。

如果 1500 个公开测试实例中，有 1490 个实例输出合法 routes，那么 feasibility rate 是：

$1490 / 1500$

这个指标非常关键。因为一个不可行解的 cost 往往没有意义。例如漏掉很多客户，cost 当然会变低，但这不是有效配送方案。

实践上应该先保证 feasibility 接近 100%，再追求 cost 下降。

5.3 Average Gap to a Reference Baseline

Gap 衡量我们的方案比某个参考 baseline 差多少或好多少。常见写法是：

$$\text{gap} = (\text{our_cost} - \text{baseline_cost}) / \text{baseline_cost}$$

如果乘以 100%，就是百分比 gap。

例如 baseline cost 是 10.0，我们的 cost 是 10.8，则：

$$\text{gap} = (10.8 - 10.0) / 10.0 = 0.08 = 8\%$$

如果我们的 cost 是 9.7，则 gap 是 -3%，表示比 baseline 更好。

本项目的训练和验证集自带 OR-Tools 参考 cost，可以用来分析训练/验证表现。公开测试和隐藏测试没有 cost，因此我们需要自己实现评估工具时，主要在验证集上比较，公开测试则主要生成符合格式的 routes。

5.4 Average Inference Time

Inference time 是生成单个实例解所需的平均时间。

它会影响提交代码在隐藏测试集上的运行表现。一个很慢的方法即使 cost 好，也可能在实际评测中不稳定。

因此 baseline 和后续 AI 方法都需要注意：

- 不能对每个实例做过长时间搜索。
- CVRP-100 的运行时间要单独关注。
- solve.py 应该能批量处理输入文件中的全部实例。

6. 为什么第一步应该先做 baseline

虽然题目强调 AI-based end-to-end solver，但第一步仍然应该先做 baseline。

原因有四个。

6.1 baseline 能验证数据读取是否正确

在训练模型前，我们必须先确认：

- .pkl 能正确读取。
- tuple 长度 6 和长度 4 都能处理。

- depot、loc、demand、capacity 的形状理解正确。
- 1-based 和 0-based 编号转换正确。

baseline 是最直接的验证方式。只要 baseline 能跑完整个公开测试集并输出合法 JSON，就说明数据流基本打通。

6.2 baseline 能验证输出格式是否正确

最终提交不是交模型训练日志，而是交可执行推理代码和 JSON 输出。

官方命令是：

```
python solve.py \  
  --input /path/to/check_data.pkl \  
  --output /path/to/predictions.json \  
  --device cuda:0 \  
  --seed 2026
```

JSON 必须包含：

```
{  
  "format_version": "cvrp_v1",  
  "solutions": [  
    {  
      "instance_id": 0,  
      "routes": [[1, 7, 3], [2, 5, 4, 6]]  
    }  
  ]  
}
```

baseline 可以先保证这个接口完全正确。

6.3 baseline 是报告里的比较对象

报告需要说明方法设计、实验和分析。如果只有一个复杂模型，没有简单 baseline，很难说明模型到底改进了什么。

一个合理 baseline 可以回答：

- 简单贪心方法能做到什么水平？
- AI 方法是否真的改善了 cost？
- AI 方法是否牺牲了 feasibility 或 inference time？
- 方法在 CVRP-50 和 CVRP-100 上是否表现一致？

6.4 baseline 是最终提交的安全兜底

神经模型训练可能出现很多问题：

- 训练时间不够。
- loss 下降但 routes 不合法。
- 模型在 CVRP-50 上能用，在 CVRP-100 上泛化差。
- 推理很慢。

如果 baseline 先完成，就至少有一个可提交、可解释、可验证的版本。后续 AI 方法可以作为增强，而不是把全部风险压在模型上。

7. Baseline 可以怎么设计

baseline 的目标不是一开始就追求最优，而是先生成稳定合法的 routes。建议从简单到稍复杂逐步做。

7.1 容量感知最近邻

思路：

1. 从 depot 出发。
2. 在未访问客户中，选择离当前位置最近且加入后不超 capacity 的客户。
3. 如果没有任何客户能加入当前 route，就结束当前 route，回到 depot，开启新 route。
4. 重复直到所有客户都被访问。

这个方法的优点：

- 实现简单。
- 推理速度快。
- 很容易保证 capacity。
- 能自然处理 CVRP-50 和 CVRP-100。

缺点：

- 容易陷入局部最优。
- 只看最近点，不考虑后续整体结构。

7.2 贪心插入

思路：

1. 先创建空 route。
2. 每次选择一个未访问客户，插入到某条 route 的某个位置。
3. 插入时要求不超 capacity。
4. 选择使 cost 增加最少的位置。

这个方法通常比简单最近邻更关注全局路径结构，但实现复杂度也更高。

7.3 route split

一种常见思路是先得到一个客户排列，再按 capacity 切分成多条 route。

例如先得到排列：

[8, 3, 10, 2, 5, 4, 6]

然后从左到右累加 demand：

[8, 3, 10] | [2, 5, 4, 6]

每段 demand 不超过 capacity。

这种思路很适合和神经网络结合。模型可以先学习输出客户访问顺序，后处理再保证 capacity。

7.4 2-opt 局部优化

2-opt 是路径内部优化方法。它不改变客户集合，只尝试交换 route 中边的连接方式，使单条 route 更短。

例如一条 route 中存在交叉路径时，2-opt 可能通过反转中间一段顺序降低距离。

优点：

- 不改变 route 的 demand，所以不会破坏 capacity。
- 可以作为最近邻或贪心后的改进步骤。

限制：

- 主要优化单条 route 内部顺序。
- 不负责把客户从一条 route 移到另一条 route。

7.5 本项目建议的 baseline 顺序

推荐顺序：

1. 先实现 capacity-aware nearest neighbor。
2. 加 feasibility checker 和 cost calculator。
3. 在验证集上和参考 cost 比较。
4. 加 route 内部 2-opt。
5. 再考虑更复杂的插入或跨 route 改进。

这样每一步都有独立价值，也便于报告中展示改进过程。

8. AI-based 方法怎么入门理解

题目鼓励使用 AI-based end-to-end 方法。这里的关键不是背模型名字，而是理解模型到底在学习什么。

8.1 Pointer Network 风格方法

Pointer Network 可以理解为“从输入客户点中一个一个指向下一个要访问的客户”。

输入是 depot、客户坐标、demand、capacity 等特征。模型每一步输出一个客户选择，逐步构造 route 或客户访问序列。

它适合 routing 问题，因为输出对象不是固定类别，而是输入节点本身。

需要注意的问题：

- 如何避免重复选择客户。
- 如何处理 capacity。
- 如何决定什么时候结束当前 route 并回 depot。
- 如何泛化到 CVRP-100。

8.2 Attention / Transformer-based routing policy

Transformer 方法会用 attention 建模客户之间的关系。

直观理解是：模型不仅看某个客户离当前位置近不近，还能综合考虑所有客户之间的空间结构、demand 分布和剩余容量。

相比 RNN, Transformer 通常更擅长处理集合结构, 因为客户点本身没有天然文本顺序。

但 Transformer 也有成本:

- 模型更复杂。
- 训练更慢。
- 需要更谨慎地设计 mask, 防止选择已访问或不可行客户。

8.3 POMO-style 强化学习

POMO 可以理解为“同一个实例从多个不同起点或策略视角同时尝试”, 通过多起点策略提升组合优化表现。

它常用于 TSP、CVRP 这类问题。核心思想是同一组客户可能有多种看起来合理的构造起点, 训练时利用这些多样性提升策略质量。

优点:

- 不完全依赖 OR-Tools 标签。
- 可能学到超过监督标签的策略。

难点:

- 强化学习训练更难稳定。
- reward、baseline、采样策略都需要设计。
- 对时间和调参要求更高。

8.4 Learning-based improvement

这种方法不是从零构造完整路线, 而是先给一个初始解, 再让模型学习如何改进它。

例如:

- baseline 先生成 routes。
- 模型判断哪些客户交换、移动或重排可能降低 cost。
- 反复改进直到时间结束或没有明显提升。

优点:

- 可以建立在稳定可行 baseline 上。
- 可行性更容易维护。

缺点：

- 实现复杂。
- 需要设计局部操作。
- 推理时间可能增加。

8.5 对本项目更现实的 AI 切入点

如果时间有限，推荐先考虑轻量方法：

- 用训练集参考 routes 做 supervised imitation，让模型学习客户排序或下一节点选择。
- 或者用 baseline 生成初始解，再训练一个简单模型辅助排序。
- 把 AI 方法作为 baseline 的增强，而不是完全替代 feasibility 逻辑。

这样可以兼顾题目对 AI-based 的要求和最终提交的稳定性。

9. 推荐的项目推进路线

建议把项目拆成四个阶段。

阶段 1：理解数据和评价

目标：

- 能读取三个 .pkl 文件。
- 能区分长度 6 和长度 4 的实例。
- 能计算 route cost。
- 能检查 feasibility。
- 能在验证集上读取参考 routes 和 cost。

这一阶段完成后，应该能回答：“给我任意一个实例和 routes，我能判断它是否合法，并计算它的 total cost。”

阶段 2：可行 baseline 和 solve.py

目标：

- 实现 capacity-aware nearest neighbor baseline。
- 支持 CVRP-50 和 CVRP-100。
- 实现官方 CLI 参数：--input、--output、--device、--seed。
- 输出符合 cvrp_v1 的 JSON。

- 在公开测试集上生成完整 predictions。

这一阶段是可提交版本的核心。

阶段 3: AI-based 增强

目标:

- 基于训练集参考 routes 设计简单监督学习任务。
- 或者把模型作为客户排序/候选选择模块。
- 保留 feasibility checker 和 baseline 作为兜底。
- 在验证集上比较 AI 方法和 baseline 的 cost、gap、time。

这一阶段要注意控制风险。AI 方法可以提升表现，但不能破坏可行性。

阶段 4: 报告和展示

目标:

- 解释问题设置和数据格式。
- 说明 baseline 和 AI 方法设计。
- 展示验证集实验结果。
- 分析 failure cases。
- 说明 CVRP-100 泛化表现。
- 说明方法限制和后续改进方向。

报告里要强调：本项目不只追求低 cost，也重视 feasibility 和 inference time。

10. 常见坑检查清单

后续写代码或报告时，可以反复检查这份清单。

数据读取

- 是否把 .pkl 当成 list of instances，而不是 dict？
- 是否正确处理训练/验证数据的长度 6 tuple？
- 是否正确处理公开/隐藏测试数据的长度 4 tuple？
- 是否注意 depot 是 $[[x, y]]$ ，不是 $[x, y]$ ？
- 是否根据 len(loc) 动态处理 CVRP-50 和 CVRP-100？

编号

- 内部数组下标是否是 0-based?
- 输出 JSON 中客户 id 是否是 1-based?
- 是否避免输出客户 0?
- 是否避免在 CVRP-50 中输出超过 50 的客户 id?
- 是否避免在 CVRP-100 中输出超过 100 的客户 id?

可行性

- 每个客户是否恰好出现一次?
- 是否没有漏客户?
- 是否没有重复客户?
- 每条 route 的 demand 是否不超过 capacity?
- depot 是否没有出现在 route 列表里?
- routes: [] 是否只在无法生成结果时使用?

输出格式

- JSON 是否包含 "format_version": "cvrp_v1"?
- solutions 数量是否等于输入实例数量?
- instance_id 是否从 0 开始并按输入顺序递增?
- 输出文件是否写到 --output 指定路径?
- 文件是否是 UTF-8 编码?

泛化和速度

- 是否在 CVRP-100 上测试过?
- 是否写死了客户数量或 capacity?
- 是否记录平均 inference time?
- 是否避免每个实例运行过长时间?

报告分析

- 是否有 baseline 比较?
- 是否单独报告 feasibility?
- 是否说明 gap 的计算方式?
- 是否分析失败案例?
- 是否说明参考 routes 不是保证全局最优?

11. 学完这份笔记后应该能做什么

学完这份笔记后，应该具备下面几项能力：

1. 能用自己的话解释本项目是在做 CVRP，而不是普通分类、回归或路径可视化任务。
2. 能说明 depot、loc、demand、capacity、routes、cost 分别是什么。
3. 能区分训练/验证数据和公开测试数据的 tuple 结构差异。
4. 能判断一组 routes 是否满足 CVRP 可行性规则。
5. 能解释为什么 feasibility 是第一优先级。
6. 能理解 cost、gap 和 inference time 分别衡量什么。
7. 能说明为什么要先做 baseline。
8. 能描述 capacity-aware nearest neighbor、route split 和 2-opt 的基本思路。
9. 能初步理解 Pointer Network、Transformer routing policy、POMO 和 learning-based improvement 在学什么。
10. 能为下一阶段 baseline、solve.py 和评估工具的实现做准备。

下一步最合理的工程目标是：先写评估工具和可行 baseline，再把 baseline 接入官方 solve.py 接口。这样项目会先获得一个稳定可提交版本，再考虑 AI-based 方法的增强。